UNIVERSITÀ DI PISA

Computer Engineering

Internet of Things

" SmartCharge: Intelligent EV Charging Management System"

Academic Year: 2024/2025

Author:

Gabriele Pianigiani

Summary

1 Introduction 3
2 Use Case: EV Charging Station with Scheduling 3
2.1 System Overview 4
2.2 System Workflow 4
3 System Implementation 4
3.1 System Components and Interaction 4
4 Data Encoding 7
4.1 JSON and SenML 7
4.2 SenML Example 7
5 Machine Learning Model 8
5.1 Model Training and Integration 8
5.1.1 Preprocessing 8
5.1.2 Training 9
5.1.3 Discretized Evaluation 9
5.1.4 Node Deployment 9
6 MySQL Database Schema 9
6.1 Table: nodes 9
6.2 Table: energy 10
6.3 Table: vehicles 10
7 Issues 10
8 Conclusions 11

1 Introduction

The increasing popularity of electric vehicles (EVs) is reshaping the modern landscape of transportation and energy. The number of EVs is growing, and with it also the demand for reliable and energy-efficient charging infrastructures. Traditional charging systems often fail to balance energy distribution effectively, especially when multiple users with different needs compete for limited charging resources.

To address this issue, **SmartCharge** proposes a dynamic scheduling system that optimizes the usage of renewable energy for EV charging. With a fixed number of charging points and a high variability in user urgency and battery status, intelligent management is essential. Vehicles with lower battery levels or higher priority should be scheduled first, maximizing fairness, energy usage efficiency, and user satisfaction.

SmartCharge presents the following objectives:

- Prioritize charging based on **battery level** and **user-defined priority**.
- Adapt to the **intermittency of renewable energy sources**.
- Minimize the dependency on energy storage solutions (like batteries) by aligning **energy production and consumption in real time**.

2 Use Case: EV Charging Station with Scheduling

The SmartCharge system (Figure 1) manages a charging station with 5 charging slots, connected to a renewable power source, particularly a wind power source. The system decides whether an EV can be charged, or if its charging can be postponed based on **real-time data** from sensors, a **machine learning prediction model** and the priority of the EV.

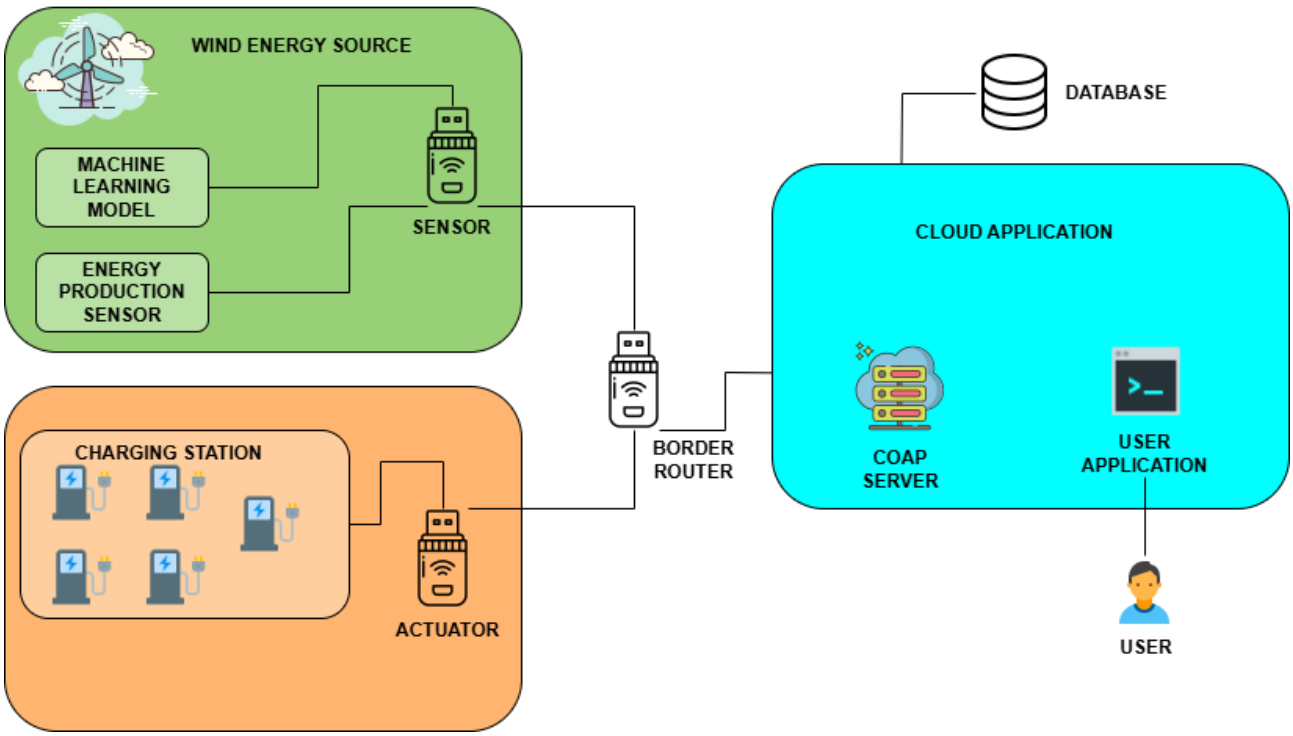

Figure 1 - SmartCharge system overview

2.1 System Overview

The architecture is divided into two main areas: **Energy Area** and **Charging Area**.

2.1.1 Energy Area

This area includes:

- A renewable energy generator, in particular a wind turbine.
- An energy sensor to monitor the energy production over time.

The sensor also includes a **machine learning model** to predict upcoming energy availability. This data helps the scheduler determine **which EVs to charge**, and when, based on available power.

2.1.2 Charging Area

This zone contains:

- 5 EV charging slots.
- An actuator which implements a scheduling controller that manages priority queues and activate or deactivate the charging of an EV based on the scheduling result.
- Sensors on each slot that read the battery level of connected vehicle. EVs also have a user-defined priority: **High** (urgent/fast charge) or **Low** (can wait).

2.2 System Workflow

Each operational cycle includes the following steps:

1. The sensor samples the current energy production.
2. The sensor also predicts near-future production using a machine learning model.
3. Collect battery levels and priorities from connected EVs when a new EV arrives.
4. Use a scheduling algorithm to assign available energy to the most critical vehicles.
5. The actuator activates/deactivates charging slots accordingly.

3 System Implementation

The SmartCharge system uses a wireless sensor network (WSN) to minimize energy consumption and allow distributed communication among energy sensor, charging units, and the central actuator.

3.1 System Components and Interaction

Each area is managed by dedicated nodes:

- **Sensor**: Monitors the energy and makes predictions.
- **Actuator**: Handles charging slot assignments.
- **Border Router**: Enables WSN-cloud application communication.
- **CoAP Server**: Handles node registration and data exchange.
- **User App**: Allows users to change priorities, monitor the slot status, see the registered nodes and so on.

3.1.1 Sensor

The sensor exposes **energy** and **settings** as resources, the first one is an observable resource, it is observed by the actuator to implement the scheduling and by the CoAP server to register the energy values in the database.

The second resource is not observable, and it is used to change the **sampling period** of the sensor, by default this value is **15 minutes**.

At the beginning the sensor registers into the CoAP server and requests the current timestamp to synchronize with the server, once the timestamp is received periodically updates it internally.

The sensor sleeps for "`sampling_period`" minutes, during this phase the **red led** is on, after this period it samples the energy and predicts the energy produced in the next "`sampling_period`" minutes (15 minutes by default), during this phase the **green led** is on.

When the energy values are computed, the sensor notifies all the observers.

The first sampled value of energy is assumed to be 5Kwh and for the following values to simulate the real sampling of energy a simple **correction factor** is applied to the predicted energy in the previous cycle.

To compute the sampled value the following operation has been implemented:

```
curr_prod = prev_prod * (1.0f + random_correction(0.10f));
```

Where:

- `prev_prod` is the predicted energy with the ML at the previous cycle.
- `random_correction` generates a random percentage variation between -10% and +10%.

3.1.2 Actuator

The actuator exposes **status** and **settings** as resources, the first one is an observable resource, it is observed only by the CoAP server to register the station status in the database.

The second resource is not observable, and it is used to allow the user to change the priority of an EV, this resource is an array of 5 elements initialized with -1 at the beginning.

At the beginning the sensor registers into the CoAP server and requests to the server the discovery of the sensor from which to observe the measured energy.

When the actuator receives an update of the resource energy or when an EV is added/removed, it computes the scheduling and updates the status of the station.

After this phase the actuator notifies the observer.

The actuator presents also an interaction with the button:

- **Button pressed for less than three seconds**: a new EV is added in the station if there is almost one free slot.
- **Button pressed for more than three seconds**: remove all EV that has completed the charge, if they are not present, one random EV is removed, no EV are removed if the station is empty.

These interactions have been introduced to simulate a realistic arrive/departure of EVs.

After the scheduling phase, the status of the station can be seen in the colour of the leds:

- **Green**: the station is empty or all present EV are in charge.

- **Red**: all present EV are waiting for a charge.
- **Yellow**: there is almost one EV which is waiting for the charge.
- **Blue**: only EVs with high priority are charging.

3.1.2.1 Scheduler Logic

The scheduling performed follows these steps:

1. All the connected EV which battery is not 100% yet, are ordered by **priority descending** and by **battery ascending** (if wo EV has the same priority, the one with small battery has more priority because it will take more time to charge).
2. For simplicity each EV has a **battery capacity** of 10Kwh and at each schedule session is charged of 1.5Kwh.
3. If there is sufficient energy to charge an EV and it has high priority, it is charged, and the available energy is reduced by the amount of energy needed to charge the EV in one step.
4. If there are only EV with low priority, the schedule is computed to don't waste time waiting for EVs with high priority.
5. If there are both high and low priority EV, and if the predicted energy is sufficient, low priority EVs will be scheduled in the future, otherwise they will be scheduled in the current session if the available energy is sufficient (the predicted energy is not reduced by the energy needed for EVs because, this will be the problem in the next schedule session).

3.1.3 Border Router

The border router facilitates remote access and control of the system, enabling the communication between local sensors and cloud-based applications.

3.1.4 CoAP Server

The CoAP server is implemented in Python and exposes as resources the **clock**, the **discovery** and the **registration**.

It handles the initial registration of the nodes as they join the network, and the discovery process when the actuator wants to find the sensor.
The server collects and stores data of energy production and the status of the station in a MySQL database, this is achieved with observing each main resource of a new registered node.

The system uses the **Constrained Application Protocol** (CoAP) instead of MQTT to better support local decision-making and time-critical control operations. Unlike MQTT, which relies on a centralized broker typically hosted in the cloud, CoAP enables direct communication between IoT nodes. This is important because in this system, actuation decisions must be made locally by the sensor node based on the current time and predicted energy availability.
By removing the need for a broker, the system achieves faster response times and meets real-time requirements essential for effective demand response. Furthermore, CoAP's decentralized nature improves system resilience, allowing nodes to continue functioning and exchanging data even during temporary failures at the border router or network gateway.

3.1.5 User Application

The user application is implemented in Python and allows the dynamic control of some parameters of the nodes and to see the status of the charging station.
The following commands are available:

- `status`: show the status of all slots.

- `show-nodes`: show all registered nodes.
- `set-priority <id> <0/1>`: set priority for a vehicle.
- `get-priority <id>`: show priority (all or single if the id is specified).
- `get-sampling-period`: show the sampling period.
- `set-sampling-period <15|30|45|60>`: set the sampling period.
- `get-energy`: show energy logs (last 24 hours).
- `exit`: exit the program.

4 Data Encoding

SmartCharge uses **JSON** for data encoding, this choice is driven by its simplicity, human readability, and widespread adoption, making it an ideal format for data interchange in IoT environments. The JSON has been chosen to encode the status data, since it does not represent a measurement, but the status of the station in terms of EVs, their batteries, priorities and state.

For the sensor measurements of energy instead, it is adopted the **SenML** (Sensor Measurement Lists) standard to ensure interoperability and seamless integration with third-party applications.

4.1 JSON and SenML

The choice of JSON data encoding is justified by the following aspects:

Simple: the JSON format is simple to generate and easy to read for a human operator, this is a good choice for an IoT environment that needs continuous monitoring and debugging.

Lightweight: in an IoT environment where the computational resources are limited having a lightweight structure increase efficiency

Widely used: thanks to the widespread diffusion, the JSON format helps the integration of the system with other third-party application.

4.2 SenML Example

The SenML has the following structure:

Base fields:

- **Base Name (bn):** string that is inserted before the names found in the entries.
- **Base Unit (bu):** a base unit that is assumed for all entries, unless otherwise indicated. If a record does not contain a Unit value, then the Base Unit is used.

Regular fields:

- **Name(n):** name of the sensor or parameter. When appended to the Base Name field, this must result in a globally unique identifier for the resource.
- **Time(t):** time when the value was recorded.
- **Unit (u):** unit for a measurement value.
- **Value (v):** values are represented using basic data types, in this case only float numbers are used.

An example of a *sensor* SenML packet is the following:

```
Sensor: Sending data:

[
```

{"bn":"Sensor:energy:","bu":"Kwh"},
{"n":"predicted","u":"Kwh","t":"2025-06-02T13:14Z","v":16403},
{"n":"sampled","u":"Kwh","t":"2025-06-02T13:14Z","v":15451}
]

Due to the limitation of the NRF52840 microcontrollers is not possible to print floating point, despite the dongle support the floating-point operations.

To resolve this, it is decided to multiply every float number by 10000 and to convert it in an integer right before the data is sent. The cloud application reconverts it to float after the reception. This give space for only four decimal numbers, but they were considered sufficient for the application.

5 Machine Learning Model

The energy prediction is powered by a **Random Forest Regressor**, trained to forecast wind output with 15 minutes granularity.

To clean, transform, and prepare the dataset for effective model training while maintaining compatibility with low-complexity inference environments.

5.1 Model Training and Integration

The sensor node uses the real time to predict the wind production of a turbine during an entire year with 15 minutes precision. The original dataset has an hourly granularity; to obtain a more consistent temporal resolution the dataset is resampled to 15-minute intervals, a linear interpolation has been implemented to fill missing values. This has been done to avoid the system produce the same value of energy between two consecutive hours.

The dataset that has been utilized is Wind Power Generation Data - Forecasting from Kaggle.

5.1.1 Preprocessing

The dataset includes data of some years, after the resampling phase only data up to 2018-01-01 is retained to reduce dimensionality and overfitting, only data of one year are considered.

A lot of attributes are removed, only the timestamp is used as parameter of the model, **month** and **day** are extracted as separate features.

A **HourDecimal** feature is computed to represent time continuously (e.g., 01:15 → 1.25).

Data between **2 AM and 7 AM** is excluded to reduce the dataset as much as possible, in this case is assumed that the station is closed between 2AM and 7AM.

A new target column, **TargetPower**, is created by shifting the power value by one step forward (i.e., 15 minutes into the future) to predict the energy produced. Any rows containing NaN values are dropped.

This preparation ensures a clean and consistent dataset, with time-based features that are meaningful predictors of wind production and avoids unnecessary data complexity that would hinder deployment on edge devices.

5.1.2 Training

The features are standardized using **StandardScaler** to ensure they are on similar scales, which helps improve model performance and convergence.

The dataset is split into training and test sets with an 80/20 ratio for performance evaluation.

Only minimal and interpretable features are selected to keep the model small and generalizable, which is essential for TinyML deployment.

A **Random Forest Regressor** is trained with:

- `n_estimators=5`: fewer trees to reduce model size.
- `max_depth=12`: limited depth to control complexity.

Random Forest provides robust predictions with relatively low computational cost. A smaller tree ensemble is chosen to reduce the memory and inference load on embedded systems.

The evaluation metrics include:

- **Mean Squared Error (MSE)** to measure prediction error.
- **R^2 Score** to assess how well the model explains the variance in data.

In this case the values are respectively: 0.0060 and 0.9226.

5.1.3 Discretized Evaluation

Both actual and predicted values are categorized into **bins**., the accuracy is measured based on how often the predicted bin matches the actual one.

Binned accuracy provides a coarse-grained but intuitive view of performance, which is useful in applications where approximate power levels are sufficient. The binned accuracy reached is about 0.79.

5.1.4 Node Deployment

The model is converted to **C code** using `emlearn`, specifically to a .h header file format.

This enables deployment of the model on microcontrollers without requiring a full Python or ML runtime.

6 MySQL Database Schema

Data is stored for analysis, visualization and configuration.

6.1 Table: nodes

In this table are registered all the nodes with the respective information.

Field	Type	Key
ip	varchar(255)	PK
name	varchar(255)	
resource	varchar(255)	
settings	varchar(255)	

Table 1 - nodes table

The attributes in Table 1 are:

- **ip**: ip address of the registered node.
- **name**: name of the node (sensor/actuator).
- **resource**: name of the observable resource exposed by the respective node.
- **settings**: settings parameter of the node.

6.2 Table: energy

This table reports the measured and predicted values of energy in time.

Field	Type	Key
timestamp	TIMESTAMP	PK
predicted	FLOAT	
sampled	FLOAT	

Table 2 - energy table

The attributes in Table 2 are:

- **timestamp**: the timestamp of the measurement.
- **predicted**: value of the predicted energy during that timestamp.
- **sampled**: value of the sampled energy during that timestamp.

6.3 Table: vehicles

Field	Type	Key
uid	INT	PK
id	INT	
battery	INT	
priority	INT	
charging	INT	

Table 3 - vehicles table

The attributes in Table 3 are:

- **uid**: it is a unique identifier, introduced because a free slot has -1 as id.
- **id**: identifier of the EV, is -1 if the slot is free.
- **battery**: level of battery of the EV.
- **priority**: priority level of the EV (1 = High, 0 = Low).
- **charging**: indicates if the EV is in charge or not (1 = yes, 0 = no).

7 Issues

After several minutes of system operation, a malfunction is observed in the actuator node. As shown in Figure 2, the actuator initially behaves as expected: it receives updates from the energy resource and reacts accordingly. However, shortly after receiving a valid update, it unexpectedly receives an **empty CoAP notification**. From that moment on, the actuator **stops observing the energy resource** and no longer updates its internal status automatically.

```
------------------------------------------------------- CONTATORE: 21
[INFO: App       ] Actuator: Notification received: 0x7de358a5fc60
Received payload: [{"bn":"Sensor:energy:","bu":"Kwh"},{"n":"predicted","u":"Kwh","t":"2025-06-02T19:40Z","v":31547},{"n":"sampled","u":"Kwh","t":"2025-06-02T19:40Z","v":31673}]
[INFO: App       ] Actuator: received sampled energy: 3.167300
[INFO: App       ] Actuator: received predicted energy: 3.154700
[INFO: App       ] Actuator: NEW-EVENT (energy sampled or button pressed)
[INFO: App       ] Scheduling process started
Residual energy: 66.33 kWh
Charging slots: [0, 0, 0, 0, 0]
[INFO: App       ] Actuator: send status: {"vehicles":[{"id":-1,"battery":-1,"priority":-1,"status":0},{"id":-1,"battery":-1,"priority":-1,"status":0},{"id":-1,"battery":-1,"priori...
[INFO: App       ] Actuator: ENTERED energy_callback. Flag: 3, Notification ptr: 0x7de358a5fc60
------------------------------------------------------- CONTATORE: 22
[INFO: App       ] Actuator: Notification received: 0x7de358a5fc60
[INFO: App       ] Actuator: Payload is NULL
Received payload:
[INFO: App       ] Actuator: Error response code
```

Figure 2 - actuator problem

Interestingly, the actuator does not crash. It remains operational, still responding correctly to commands issued from the application interface. Furthermore, if the physical button is pressed, the actuator is able to update the status manually. This confirms that the node itself is not blocked or frozen, but specifically that the CoAP observation mechanism has silently failed.

The issue does not originate from the sensor node. The sensor continues to send periodic updates for the energy resource to its active observers. Network inspection confirms that only one observer remains (the CoAP server) indicating that the actuator is no longer subscribed.

Several attempts were made to mitigate the issue. For instance, a check was added to detect if a CoAP notification with a reset flag (flag 3) was received, in which case the actuator would explicitly cancel and re-establish the observation. Despite this, the problem persists.

Further analysis revealed a striking regularity: in the absence of application interactions, the issue consistently occurs **on the 22nd call to the energy_callback function**. When the actuator is also receiving requests from the application, the failure tends to happen slightly earlier—around the 18/20 callback. This pattern led to the hypothesis that the problem could be related to **CoAP transaction buffer exhaustion**.

CoAP uses a limited-size buffer to manage open transactions. If this buffer becomes saturated, because old transactions are not acknowledged or cleared in time, new messages may be dropped or mishandled. This could explain why a notification is received as "empty" and why observation is silently lost. To test this theory, the `COAP_MAX_OPEN_TRANSACTIONS` value was increased in the configuration. However, this change did **not** resolve the issue, suggesting that the problem may be more subtle, possibly involving timing, acknowledgment delays, or internal state inconsistencies in the CoAP implementation.

8 Conclusions

SmartCharge introduces an intelligent, scalable, and energy-aware solution for managing electric vehicle charging. By combining machine learning-based energy prediction with a robust, priority-driven scheduling algorithm, the system enhances both energy resource utilization and the overall user experience. Its architecture is designed to operate autonomously and adaptively, making it well-suited for deployment in environments with fluctuating energy availability and varying user demands.

Looking ahead, several enhancements could significantly extend the system's capabilities. Integrating battery storage systems would help buffer supply fluctuations, ensuring stable operation even under intermittent renewable generation. The introduction of dynamic pricing models, adjusted based on real-time demand levels or user urgency, could further incentivize energy-efficient behaviour and optimize grid load balancing. Additionally, coupling the system with weather forecasting APIs would improve predictive accuracy, especially in renewable energy contexts. Finally, scaling the scheduling algorithm to efficiently handle dozens or even hundreds of charging slots would allow SmartCharge to support larger infrastructures, such as urban charging stations.

Overall, SmartCharge represents a meaningful step toward the next generation of green mobility infrastructure. It offers a practical and sustainable approach to aligning energy availability with real-time consumption, ultimately contributing to a smarter and more resilient energy ecosystem.